

# Résumé d'examen

## Calculabilité et complexité

*Synthèse orientée partiel final - basée sur le cours et le TD fournis*

Objectif: tenir en moins de 10 pages ce qu'il faut surtout savoir refaire ou expliquer à l'examen: définitions, théorèmes, schémas de preuve, classes de complexité et méthodes de réduction.

### **Très probable**

Machine de Turing, calculabilité, langage reconnu/décidable, temps/espace, machine universelle, indécidabilité de HALT/INC.

### **À savoir appliquer**

P, NP, PSPACE, certificats, non-déterminisme, réduction polynomiale, logique d'une preuve de NP-complétude sur un jeu/problème.

## 1. Ce qu'il faut absolument maîtriser

- Donner la définition complète d'une machine de Turing: alphabet, ruban, tête de lecture, ensemble d'états, transitions, états d'acceptation et de rejet.
- Distinguer précisément langage reconnu, langage décidable et machine qui ne termine pas sur certaines entrées.
- Calculer ou estimer une complexité temps/espace d'une MT simple, par exemple le palindrome.
- Expliquer une simulation de modèle de calcul: changement d'alphabet, passage de plusieurs rubans à un seul, ruban bi-infini vers ruban infini dans un seul sens.
- Connaître l'idée d'encodage des MT, la machine universelle et la diagonalisation utilisée pour montrer qu'un langage n'est pas reconnaissable.
- Savoir présenter HALT et INC/INC $\bar{C}$  et comprendre quelle hypothèse mène à la contradiction.
- Connaître les classes P, NP, PSPACE, le rôle du non-déterminisme et la caractérisation de NP par certificat.
- Savoir définir une réduction polynomiale  $A \leq_p B$  et expliquer pourquoi B dans P implique alors A dans P.
- Être capable de décrire la structure d'une preuve de NP-complétude à partir d'un problème connu comme SAT ou 3-SAT.

| Thème         | Question type examen                                         | Réflexe à avoir                                                                      |
|---------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Calculabilité | "Pourquoi ce langage n'est pas décidable / reconnaissable ?" | Chercher un encodage des MT + contradiction par machine universelle/diagonalisation. |
| Complexité    | "Dans quelle classe est ce problème ?"                       | Vérifier temps polynomial, espace polynomial, certificat, ou réduction.              |
| TD            | "Expliquez le rôle du gadget / de l'état"                    | Dire ce qu'il impose localement et comment il sert la preuve globale.                |

## 2. Machines de Turing: bases à savoir expliquer proprement

Une machine de Turing est un modèle abstrait d'ordinateur. Elle lit une entrée sur un ruban, modifie éventuellement des cases, change d'état et déplace sa tête de lecture vers la gauche ou la droite selon une fonction de transition.

### Définitions essentielles

- Alphabet Sigma contenant les symboles utiles et souvent # pour la case vide.
- Ruban (souvent bi-infini dans le cours) découpé en cases.

- Tête de lecture qui pointe une case à la fois: elle ne "saute" pas directement, elle se déplace pas à pas.
- Ensemble d'états  $Q$  avec un état initial  $q_0$ , un état acceptant  $q_a$  et un état rejetant  $q_r$ .
- Transition typique:  $\delta(q, a) = (q', b, \text{direction})$ .

### Langage reconnu vs langage décidable

| Notion                           | Ce que fait la MT sur $x$ dans $L$ | Ce que fait la MT sur $x$ hors de $L$           |
|----------------------------------|------------------------------------|-------------------------------------------------|
| Langage reconnu (semi-décidable) | La machine finit par accepter.     | Elle peut rejeter, ou bien ne jamais s'arrêter. |
| Langage décidable                | La machine s'arrête et accepte.    | La machine s'arrête et rejette.                 |

### Complexité temps et espace

- Temps  $T(n)$ : nombre d'itérations avant d'atteindre un état final sur une entrée de taille  $n$ .
- Espace  $S(n)$ : nombre de cases utilisées/modifiées sur le ruban pendant le calcul.
- Toujours retenir: l'espace est en général plus petit que le temps, car utiliser  $k$  cases demande au moins  $k$  étapes.

### Exemple classique: le palindrome

- Idée: mémoriser le bit le plus à gauche, l'effacer, aller jusqu'au dernier bit non effacé, comparer, l'effacer, revenir au début et recommencer.
- Temps:  $\Theta(n^2)$ , car on balaie presque tout le mot à chaque paire comparée.
- Espace:  $\Theta(n)$ , car le ruban utilise une portion linéaire en la taille du mot.

**Piège fréquent: beaucoup d'étudiants confondent ruban parcouru et déplacement instantané. La tête revient au début en repassant case par case; ce coût compte dans le temps.**

## 3. Simulations de modèles: idées et surcoûts à retenir

Le cours insiste sur l'idée suivante: plusieurs variantes de MT ont la même puissance de calcul, mais pas exactement le même coût. À l'examen, il faut surtout savoir expliquer le principe de codage et d'où vient le facteur de surcoût.

### Changer d'alphabet

- Chaque symbole de l'alphabet  $\Sigma$  est recodé en blocs binaires de longueur environ  $\log_2(|\Sigma|)$ .
- Conséquence: une MT binaire peut simuler une MT sur  $\Sigma$  avec un surcoût multiplicatif de l'ordre de  $\log(|\Sigma|)$  en temps et en espace.

- Message à retenir: le changement d'alphabet ne change pas la calculabilité, seulement le coût par symbole simulé.

### Passage de k rubans à 1 ruban

- On code le contenu des k rubans sur un seul ruban, en marquant la position des k têtes.
- Pour simuler un pas de la machine à k rubans, la machine à 1 ruban doit balayer la zone utile pour lire les symboles sous les têtes, puis balayer encore pour écrire et déplacer les marqueurs.
- Résultat important du cours: si la machine à k rubans tourne en temps  $T(n)$  et espace  $S(n)$ , alors la simulation à 1 ruban prend typiquement  $\Theta(k T(n)^2)$  en temps et  $\Theta(k S(n))$  en espace.

### Ruban bi-infini vers ruban infini dans un seul sens

- Idée standard du TD: réencoder les positions négatives et positives d'un ruban bi-infini sur un ruban indexé par  $N$ .
- Le TD demande de montrer qu'on garde la reconnaissabilité avec un facteur 4 en temps: c'est un bon exercice de simulation de modèle.

Schéma de réponse efficace: 1) dire comment on code l'information, 2) dire comment on simule un pas, 3) compter le coût d'un pas simulé, 4) multiplier par le nombre total de pas.

## 4. Encodage des machines, machine universelle et indécidabilité

Comme une MT est décrite par une quantité finie d'information, on peut l'encoder par un mot binaire ou par un entier. Cela permet de numérotter les machines  $M_0, M_1, M_2, \dots$  puis de construire une machine universelle  $U$ .

- Entrée de la machine universelle:  $\langle x, w \rangle$  où  $x$  code une machine  $M_x$  et  $w$  est un mot.
- Comportement:  $U$  simule  $M_x$  sur l'entrée  $w$  et reproduit son résultat.
- C'est l'outil de base pour parler du langage HALT et pour faire des preuves par contradiction.

### Langage diagonal / INC

- Le cours introduit un langage de type INC / INC̄ construit à partir du comportement de  $M_x$  sur sa propre description  $x$ .
- L'idée clé: supposer qu'une MT reconnaît ce langage, puis regarder ce qu'elle fait sur sa propre entrée. On obtient un paradoxe du type "elle accepte si et seulement si elle n'accepte pas" ou une contradiction équivalente.
- Conclusion à retenir: il existe des langages qui ne sont pas reconnaissables par machine de Turing.

### HALT

- Définition:  $\text{HALT} = \{ \langle x, y \rangle \mid \text{la machine } M_x \text{ s'arrête sur l'entrée } y \}$ .
- On montre par l'absurde que HALT n'est pas décidable. Souvent on suppose l'existence d'une machine  $H$  décidant HALT, puis on l'utilise comme sous-routine pour construire une machine diagonale impossible.

- Le schéma standard: tester si  $Mx$  s'arrête sur  $x$ , puis faire l'inverse de ce que prédit  $H$ ; la contradiction prouve que  $H$  ne peut pas exister.

**Réflexe examen**

Dès qu'on voit 'sur sa propre description', 'sur l'entrée  $x$ ', 'utiliser HALT comme boîte noire', penser diagonalisation / auto-référence.

## 5. Deuxième partie: complexité, non-déterminisme et certificats

### Classes déterministes

- $DTIME(f)$ : langages reconnus en temps  $O(f(n))$ .
- $DSPACE(f)$ : langages reconnus en espace  $O(f(n))$ .
- $P$  = union pour  $c > 0$  de  $DTIME(n^c)$ .
- $PSPACE$  = union pour  $c > 0$  de  $DSPACE(n^c)$ .

### Machine de Turing non déterministe

- À chaque étape, plusieurs choix de transition sont possibles.
- Le mot est accepté s'il existe au moins une branche de calcul qui mène à l'acceptation.
- $NTIME(f)$  et  $NSPACE(f)$  se définissent comme les versions non déterministes de temps et espace.
- $NP$  = union pour  $c > 0$  de  $NTIME(n^c)$ .

### Relations à retenir

| Inclusion             | Pourquoi / intuition                                                                                                                    |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| $P \subseteq NP$      | Une machine déterministe est un cas particulier de machine non déterministe.                                                            |
| $NP \subseteq PSPACE$ | On peut explorer systématiquement les choix non déterministes avec un compteur ou un certificat en n'utilisant qu'un espace polynomial. |
| $PSPACE = NPSPACE$    | Résultat annoncé dans le cours; à connaître au moins comme théorème.                                                                    |

### NP comme classe des vérifications efficaces

- Définition alternative très importante: un langage  $L$  est dans  $NP$  s'il existe une MT déterministe qui, sur entrée  $x$  et certificat  $c$ , vérifie en temps polynomial que  $x$  appartient à  $L$ .
- Autrement dit: pour les instances "oui", il existe une preuve courte et vérifiable rapidement.
- C'est exactement l'idée utilisée pour montrer qu'un problème est dans  $NP$ : proposer quel certificat on devine et comment on le vérifie.

Exemple de formulation propre: 'Le certificat est ... ; sa taille est polynomiale ; la vérification consiste à ... ; elle se fait en temps polynomial ; donc le problème est dans NP.'

## 6. Réductions polynomiales et logique d'une preuve de NP-complétude

Une réduction polynomiale de A vers B, notée  $A \leq_p B$ , est une fonction f calculable en temps polynomial telle que x appartient à A si et seulement si f(x) appartient à B.

- Interprétation: si B est facile (dans P), alors A est facile aussi, car on transforme x en f(x) puis on résout B.
- Transitivité: si  $A \leq_p B$  et  $B \leq_p C$ , alors  $A \leq_p C$ .
- Usage standard à l'examen: pour prouver qu'un problème B est NP-dur, on part d'un problème déjà NP-complet A et on montre  $A \leq_p B$ .

### Plan canonique d'une preuve de NP-complétude

- Montrer B dans NP: donner un certificat et la vérification en temps polynomial.
- Choisir un problème A déjà NP-complet (souvent SAT ou 3-SAT).
- Décrire la transformation  $\phi \rightarrow I(\phi)$  vers une instance de B.
- Prouver que la taille de I(phi) est polynomiale en la taille de phi.
- Prouver le sens 1: si phi est satisfiable, alors I(phi) est une instance oui de B.
- Prouver le sens 2: si I(phi) est une instance oui de B, alors phi est satisfiable.
- Conclure: B est NP-dur; avec B dans NP, on obtient B NP-complet.

### Question ouverte centrale

Le cours rappelle que la grande question reste  $P = NP$  ? On ne sait pas si tout problème dont les solutions se vérifient vite peut aussi se résoudre vite.

## 7. Ce que le TD suggère comme savoir-faire

### TD Machines de Turing

- Faire tourner une MT sur une entrée donnée: suivre état par état, ruban d'entrée, ruban de travail et positions des têtes.
- Expliquer le rôle de chaque état: phase de copie, retour à gauche, comparaison, nettoyage, acceptation, rejet, etc.
- Déterminer le langage reconnu à partir du comportement global de la machine.
- Concevoir une MT qui écrit  $n^2$  bits ou  $2^n$  bits: penser en termes de compteurs, boucles imbriquées et duplication d'information.
- Montrer une simulation avec ruban infini dans un seul sens: exercice typique de codage et de majoration du temps.

**TD NP-complétude / gadgets**

- Quand un jeu apparaît (Cubic, Pokémon, Phutball), le but n'est pas de mémoriser tous les dessins mais de comprendre la logique de réduction.
- Pièce variable: force un choix exclusif vrai/faux pour chaque variable.
- Pièce clause: permet d'ouvrir un passage si au moins un littéral de la clause a été activé par le choix de variable.
- Gadgets de croisement, switch, sens unique, fusible: ils servent à contrôler les interactions locales entre chemins sans casser la structure globale de la réduction.
- Dans une preuve complète, il faut toujours justifier trois choses: polynomialité de la construction, correction dans le sens "oui", correction dans le sens réciproque.

**Comment commenter un gadget à l'oral ou à l'écrit**

- Dire quelle entrée et quelle sortie il autorise.
- Dire ce qu'il interdit (retour arrière, double passage, choix multiple, croisement libre, etc.).
- Relier son effet local à la signification logique (variable, clause, propagation, verrouillage...).

**8. Fiche mémoire ultra-courte**

| Mot-clé                      | À réciter presque tel quel                                                                                                      |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Reconnaissable</b>        | La machine accepte tous les mots du langage; hors du langage elle peut rejeter ou boucler.                                      |
| <b>Décidable</b>             | La machine s'arrête toujours: accepte dans le langage, rejette hors du langage.                                                 |
| <b>Machine universelle</b>   | Elle simule $M_x$ sur $w$ à partir de l'entrée $\langle x, w \rangle$ .                                                         |
| <b>HALT</b>                  | $\text{HALT} = \{ \langle x, y \rangle \mid M_x \text{ s'arrête sur } y \}$ ; non décidable par diagonalisation.                |
| <b>P</b>                     | Problèmes résolus en temps polynomial par une machine déterministe.                                                             |
| <b>NP</b>                    | Problèmes dont une solution peut être vérifiée en temps polynomial, ou résolus par une MT non déterministe en temps polynomial. |
| <b>Réduction polynomiale</b> | $x$ dans $A$ ssi $f(x)$ dans $B$ , avec $f$ calculable en temps polynomial.                                                     |
| <b>NP-complet</b>            | Dans NP et NP-dur.                                                                                                              |

### Derniers conseils

- Toujours distinguer puissance de calcul (ce qui est calculable) et coût (temps/espace).
- Quand on vous demande "pourquoi non décidable ?", ne racontez pas juste HALT: dites précisément quelle hypothèse vous faites et où naît la contradiction.
- Quand on vous demande une complexité, comptez les balayages complets du ruban; c'est souvent là que se cache le  $n^2$ .
- Quand on vous demande une NP-complétude, structurez la copie avec des sous-titres: NP, transformation, taille, sens aller, sens retour.